

VOLUMEN II · LIBRO 2 DE 5

BMAD Flujos de Proyecto

El recorrido completo, fase a fase, para proyectos nuevos y existentes



Cómo se encadenan los workflows del método.
Greenfield y brownfield, paralelismos, dependencias y roles.
Acompaña al Libro 1 (Atlas) con el detalle del proceso.

BMAD Method · CIS · TEA · BMB · Core

2026-05-07 · LIDR.co

0 Bienvenida

Este es el segundo libro de la biblioteca. Asume que el lector ya conoce el vocabulario y los módulos del método (Libro 1 — Atlas). Su función es mostrar **cómo se enlazan los workflows en el tiempo**: qué viene antes, qué se puede paralelizar y qué decisiones bloquean al equipo si se saltan.

Cubre dos escenarios completos:

- **Flujo greenfield** — proyecto nuevo desde la idea inicial hasta la implementación continua.
- **Flujo brownfield** — proyecto ya iniciado al que se incorpora el método.

También recoge la tabla de paralelismos, el resumen de roles por fase, el mapeo a las sesiones operativas (Libro 3) y el conjunto de skills que *no* forman parte del flujo principal pero conviene conocer.

0.1 Cómo está organizado este libro

Sección	Contenido	Para qué la usas
Sec. 1 Convenciones	Insignias de obligatoriedad, módulos y aportes LIDR. Resumen rápido del lenguaje visual del libro.	Hojear antes de leer si es la primera vez.
Sec. 2 Flujo greenfield	Las cuatro fases secuenciales con los workflows de cada una, los roles implicados y el conocimiento que cada workflow espera.	Para proyectos que arrancan desde cero.
Sec. 3 Flujo brownfield	Documentación del proyecto existente y árbol de decisión para elegir camino según la magnitud del cambio.	Para proyectos en marcha que adoptan BMAD.
Sec. 4 Paralelismo	Qué se puede hacer en simultáneo y qué cadenas son estrictamente secuenciales.	Para planificar sesiones y distribuir trabajo.
Sec. 5 Roles	Tabla de roles del equipo, fases en las que participan y agrupaciones típicas en equipos pequeños.	Para asignar responsables.
Sec. 6 Mapeo a sesiones	Cómo encajan estas fases en el calendario S1–S6 del Libro 3.	Para preparar la agenda real de adopción.
Sec. 7 Fuera de scope	Skills que existen en el método pero no forman parte del flujo del proyecto.	Para saber dónde mirar cuando una skill no aparece en el flujo.
Sec. 8 Recomendaciones operativas	Buenas prácticas transversales: contexto persistente, ventana fresca, validación cruzada.	Para evitar errores comunes al ejecutar.

1 Convenciones

El libro reutiliza el sistema visual del Libro 1. Esta sección lo resume para uso rápido.

1.1 Punto de color del módulo

- **BMM** · flujo principal
- **TEA** · testing
- **CIS** · pensamiento creativo
- **BMB** · extensión del método
- **Core** · utilidades transversales

1.2 Insignias de obligatoriedad

- REQ** obligatoria para avanzar de fase ·
- REC** recomendada fuertemente ·
- OPC** opcional según necesidad ·
- REQ TEA** obligatoria si el equipo adopta TEA.

LECTURA DE LAS INSIGNIAS

La insignia refleja lo que el método declara en su instalación: qué workflows son obligatorios para avanzar de fase. Cuando un workflow figura como opcional pero conviene tratarlo como recomendado, lo verás marcado con la insignia

REC y, si la recomendación es nuestra, con la insignia adicional **RECOMENDACIÓN LIDR**.

1.3 Insignias de aporte LIDR

Cuando una afirmación del libro es interpretación o recomendación nuestra — no parte del método oficial — aparece marcada con una insignia.

VISIÓN LIDR framing conceptual: cómo entendemos o reorganizamos el método.

RECOMENDACIÓN LIDR sugerencia operativa basada en nuestra experiencia aplicando el método con equipos.

2 Flujo greenfield — proyecto nuevo

Recorrido completo desde la idea inicial hasta la implementación continua. Las fases son secuenciales: cada una produce los artefactos que la siguiente necesita.

CÓMO OPERA TEA EN ESTE FLUJO

El módulo de testing no es una cadena lineal única. Funciona en tres bloques con dependencias internas:

- **Arranque** en Fase 3 — test-design → framework → ci .
- **Por historia** en Fase 4 — atdd → dev-story → test-automate → code-review .
- **Por épica o hito** en Fase 4 — test-review , trace y nfr .

Dentro de cada bloque el orden es estricto. Entre bloques el orden natural es arranque → por historia → por épica.

ADOPTAR TEA ES OPCIONAL, EJECUTARLO NO

Si el equipo decide adoptar TEA, las dependencias críticas son: **(1)** el framework instalado antes del primer ATDD; **(2)** la pipeline de integración continua antes del primer merge; **(3)** los umbrales no funcionales (latencia, RPS¹, seguridad)

1. RPS: *requests per second*. Métrica de carga que indica cuántas peticiones por segundo soporta un sistema. Se usa para fijar los umbrales no funcionales del PRD junto con la latencia P95.

fijados antes del primer release. El mínimo viable es `test-design` → `framework` → `atdd`. Saltarse esto deja al equipo sin red de seguridad y la suite degradada en pocas semanas.

2.1 FASE 1 — ANÁLISIS 1–2 sesiones

Genera el contexto del producto: idea de negocio, mercado, dominio y restricciones técnicas. Toda la fase es opcional; si el equipo llega con visión cristalizada, brief sólido y research previo, puede saltarse. Cada hora invertida aquí ahorra retrabajo posterior.

Sesión / Roles	Workflow	Conocimiento que necesita
Sesión 1 (2–3 h) Roles: PO, Stakeholder	<code>/bmad-brainstorming</code> ● Brainstorming OPC	Idea de negocio, problemas a resolver, mercado al que apunta el producto.
Sesión 1 Roles: PO, Stakeholder	<code>/bmad-product-brief</code> ● Product Brief REC	Visión del producto, usuarios objetivo, alcance.
Sesión 2 (2 h) Roles: PO, experto de dominio	<code>/bmad-market-research</code> ● Market Research OPC	Competidores, regulación del sector, comportamiento del cliente.
Sesión 2 Roles: PO, experto de dominio	<code>/bmad-domain-research</code> ● Domain Research OPC	Terminología, procesos del sector, marco regulatorio.
Sesión 2 Roles: Tech Lead, DevOps	<code>/bmad-technical-research</code> ● Technical Research OPC	Restricciones técnicas, stack candidato, infraestructura disponible.

ALTERNATIVAS CREATIVAS PARA LA IDEACIÓN

`/bmad-brainstorming` es ideal cuando la idea aún no está cristalizada. Si el PO llega con visión clara, se puede saltar. Si el equipo necesita un enfoque más estructurado, hay dos opciones del módulo creativo:

`/bmad-cis-design-thinking` centra la ideación en empatía con el usuario y `/bmad-cis-innovation-strategy` aborda enfoques disruptivos. Ninguna de estas tres es prerequisite de las siguientes fases — son herramientas para activar al equipo.

CUÁNDO SALTAR LA FASE 1

Solo es prudente saltarla con tres condiciones simultáneas: brief redactado, research previo verificado y visión del producto cristalizada. Sin alguna de estas, vuelves a esta fase semanas después con coste mayor.

2.2 FASE 2 — PLANIFICACIÓN 2–3 sesiones

Produce el PRD² y, si el producto tiene UI, la especificación de UX. Es el primer hito con un workflow obligatorio: sin PRD no hay arquitectura ni épicas.

↓ continúa en la página siguiente

2. PRD: Product Requirements Document. Documento que describe lo que el producto debe hacer (requisitos funcionales) y cómo de bien debe hacerlo (requisitos no funcionales). Es el contrato entre producto y equipo técnico.

Sesión / Roles	Workflow	Conocimiento que necesita
Sesión 3 (2 h) Roles: PO, Tech Lead	/bmad-create-prd ● Create PRD REQ	Brief del producto, requisitos funcionales y no funcionales esperados.
Sesión 4 (2 h) Roles: PO, Tech Lead	/bmad-validate-prd ● Validate PRD REC	PRD terminado.
Sesión 4 Roles: UX, PO	/bmad-create-ux-design ● Create UX Design REC SI UI	PRD, flujos de usuario esperados.
Sesión 4 Roles: QA, Tech Lead, DevOps (consultor)	/bmad-testarch-nfr (setup) ● NFR Setup REC	PRD completo. Fija umbrales cuantificables (latencia P95, RPS, SLO ⁵). DevOps consulta sobre capacidad real de la infraestructura.

RECOMENDACIÓN LIDR

El método marca **/bmad-validate-prd** como opcional. Recomendamos tratarlo como paso recomendado: saltar la validación es la causa más frecuente de retrabajo en Fase 4, porque los huecos del PRD viajan al backlog y se hacen visibles cuando el equipo ya está implementando.

PRD TESTEABLE: DOS VERIFICACIONES

- **Requisitos funcionales con criterios Given/When/Then⁴** durante **/bmad-validate-prd**. Un requisito no testeable aquí se convierte en una historia no testeable en Fase 4.
- **NFR⁵ con umbrales cuantificables** a través de **/bmad-testarch-nfr** en modo setup: por ejemplo, "P95 de la API por debajo de 200 ms con 100 RPS", no "el sistema debe ser rápido". Sin baseline aquí, el gate antes del release no tiene contra qué validar.

VISIÓN LIDR

En el instalador, NFR es una sola skill con tres modos. Nosotros la pensamos como **tres momentos del proyecto**: *setup* en Fase 2 para fijar umbrales, *check* a mitad del ciclo de implementación para detectar desviaciones tempranas y *gate* antes del release como decisión final go/no-go. Esta separación facilita la conversación con el equipo y deja claro qué se evalúa cada vez.

2.3

FASE 3 — SOLUCIÓN

3–4 sesiones

Diseña la arquitectura, arranca la infraestructura de testing y descompone el alcance en épicas e historias. Es la fase con más obligatoriedad: la arquitectura, el backlog y la verificación de readiness son requeridos para pasar a implementación.

↓ continúa en la página siguiente

5. SLO: Service Level Objective. Objetivo cuantificable de calidad de servicio que el equipo se compromete a cumplir — por ejemplo, "el 99,9 % de las peticiones responde en menos de 200 ms". Sin SLO, no hay forma objetiva de saber si el sistema funciona bien.

5. Given/When/Then: estructura de criterios de aceptación que describe un escenario en tres partes — dado un contexto inicial (Given), cuando ocurre un evento (When), entonces se espera un resultado (Then). Es el formato estándar de BDD (Behavior-Driven Development).

5. NFR: Non-Functional Requirement. Requisito que describe cómo de bien debe comportarse el sistema (rendimiento, seguridad, fiabilidad, escalabilidad), en contraposición a los *functional requirements*, que describen qué debe hacer. Sin umbrales cuantificables, los NFR son intenciones sin contrato.

Sesión / Roles	Workflow	Conocimiento que necesita
Sesión 5 (2 h) Roles: Architect / Tech Lead, DevOps	/bmad-create-architecture ● Create Architecture REQ	PRD, UX, stack preferido, restricciones de infraestructura.
Sesión 6 (2 h) Roles: QA, Tech Lead	/bmad-testarch-test-design ● Test Design REQ TEA	Arquitectura y PRD para planificación basada en riesgo.
Sesión 6 Roles: QA, Tech Lead	/bmad-testarch-framework ● Test Framework REQ TEA	Test Design, stack tecnológico decidido.
Sesión 6 Roles: QA, DevOps	/bmad-testarch-ci ● CI Setup REQ TEA	Framework instalado, plataforma de integración continua.
Sesión 7 (2 h) Roles: PO, Tech Lead, SM	/bmad-create-epics-and-stories ● Create Epics & Stories REQ	PRD, arquitectura, estimaciones.
Sesión 8 (1 h) Roles: Tech Lead, PO	/bmad-check-implementation-readiness ● Check Readiness REQ	Todos los artefactos anteriores completos.

POR QUÉ TEA ARRANCA DESPUÉS DE LA ARQUITECTURA

El bloque **test-design** → **framework** → **ci** se ejecuta tras la arquitectura porque depende de las decisiones de stack. Las tres skills son secuenciales entre sí. Si el equipo no tiene QA dedicado, el Tech Lead puede asumir el bloque sin perder rigor.

GATE DE TESTABILIDAD POST-ARQUITECTURA

Antes de Test Design, conviene aplicar el *ADR Quality Readiness Checklist*: 29 criterios en 8 categorías que verifican si las decisiones arquitectónicas son testeables. ADRs⁶ ambiguos producen tests imposibles de diseñar. El detalle del checklist vive en el Libro 5 (Protocolo TEA).

API TESTING COMO PRIORIDAD

En arquitecturas con muchas APIs, la estrategia recomendada es preferir niveles bajos de testing — unitario por encima de integración, integración por encima de end-to-end. La decisión se toma al diseñar el plan, no al final, cuando ya está la suite construida. El detalle vive en el Libro 5.

CUANDO ARQUITECTURA Y PRODUCTO CHOCAN

Si la arquitectura impone restricciones que entran en conflicto con el PRD, **/bmad-cis-problem-solving** guía al equipo con metodologías estructuradas para resolver el conflicto antes de descomponer en épicas. Forzar el avance sin resolverlo arrastra el conflicto hasta el código.

GATE DE PROCESO: CHECK READINESS

6. ADR: *Architecture Decision Record*. Documento corto que captura una decisión técnica (qué se eligió, qué se descartó y por qué), con sus trade-offs. Es el rastro auditable de las decisiones de arquitectura, distinto del documento general de arquitectura.

`/bmad-check-implementation-readiness` debe terminar en estado *READY* antes de avanzar a Fase 4. Si devuelve *NEEDS WORK*, los puntos pendientes se resuelven primero. Saltarse este gate es la causa principal de retrabajo en implementación.

2.4 FASE 4 — IMPLEMENTACIÓN ciclo continuo

Es la fase más larga y la única no acotada en el tiempo: dura tantos sprints como el proyecto necesite. Se compone de cuatro bloques: arranque del sprint, ciclo por historia, cierre de épica e invocaciones puntuales del NFR.

2.4.1 ARRANQUE DEL SPRINT

Cuándo / Roles	Workflow	Conocimiento que necesita
Kick-off (2 h) Roles: SM, Tech Lead, Devs	<code>/bmad-sprint-planning</code> ● Sprint Planning REQ	Épicas, historias, capacidad del equipo.

2.4.2 CICLO POR HISTORIA — ORDEN ESTRICTO

Cuándo / Roles	Workflow	Conocimiento que necesita
1. Por historia Roles: Dev asignado	<code>/bmad-create-story</code> (<i>create</i>) ● Create Story REQ	Story spec, arquitectura, historias previas.
2. Por historia Roles: Dev asignado, Tech Lead	<code>/bmad-create-story</code> (<i>validate</i>) ● Validate Story REC	Historia recién creada. Verifica que está completa antes de pasar al desarrollo.
3. Por historia Roles: QA, Dev	<code>/bmad-testarch-atdd</code> ● ATDD REQ TEA	Historia con criterios de aceptación.
4. Por historia Roles: Dev asignado	<code>/bmad-dev-story</code> ● Dev Story REQ	Historia validada, patrones del proyecto.
5. Por historia Roles: QA	<code>/bmad-testarch-automate</code> ● Test Automation REC	Código implementado (entre Dev Story y Code Review).
6. Por historia Roles: Dev + Tech Lead	<code>/bmad-code-review</code> ● Code Review REC	Código y tests completos — gate antes del merge.

RECOMENDACIÓN LIDR El método marca `/bmad-code-review` y `/bmad-create-story` en modo *validate* como opcionales. Recomendamos tratar ambos como paso recomendado del ciclo: el *validate* intercepta historias mal formadas antes de que el agente Dev las implemente; el code review captura defectos antes del merge usando un LLM distinto al que implementó. Saltarlos abarata el ciclo a corto plazo y lo encarece a medio plazo.

CÓMO ENCAJA EL CICLO TEA POR HISTORIA

ATDD genera tests que fallan (fase roja del ciclo red-green-refactor⁷) *antes* de Dev Story. Test Automation expande la cobertura (unitario, casos de borde, regresión) *antes* del Code Review. El code review actúa como gate final viendo

7. red-green-refactor: ciclo de Test-Driven Development en tres pasos. Rojo: escribir un test que falle. Verde: implementar lo mínimo para que el test pase. Refactor: limpiar el código manteniendo el test verde. Se repite por cada incremento.

código y tests juntos. Al final de cada épica, Test Review audita la calidad de los tests acumulados y Traceability verifica la cobertura contra requisitos.

2.4.3 CIERRE DE ÉPICA

Cuándo / Roles	Workflow	Conocimiento que necesita
1. Fin de épica Roles: QA	<code>/bmad-testarch-test-review</code> ● Test Review OPC	Tests acumulados de la épica. Devuelve puntuación 0–100 y hallazgos por dimensión.
2. Fin de épica Roles: QA, Tech Lead	<code>/bmad-testarch-trace</code> ● Traceability OPC	Matriz de cobertura requisitos↔tests, decisión de gate (después de Test Review).
3. Fin de épica Roles: Todo el equipo	<code>/bmad-retrospective</code> ● Retrospective OPC	Salidas de Test Review y Traceability como entradas para lecciones y action items.

TRAZABILIDAD VIVA, NO RETROACTIVA

La matriz de requisitos contra tests se mantiene historia a historia: ATDD ata cada test a su requisito funcional al crearlo. `/bmad-testarch-trace` al cierre de épica es el gate, no la primera vez que el equipo construye la matriz. Si se deja para el final, la matriz se vuelve arqueología retroactiva: alguien intenta reconstruir qué test cubre qué requisito a partir del nombre del archivo, y se generan huecos invisibles.

2.4.4 NFR A MITAD DEL CICLO Y ANTES DEL RELEASE

Cuándo / Roles	Workflow	Conocimiento que necesita
Mid-ciclo Roles: QA, Tech Lead, DevOps	<code>/bmad-testarch-nfr</code> (check) ● NFR Check REC	Revisión intermedia. Detecta desviaciones temprano frente a los umbrales fijados en setup.
Pre-release Roles: QA, Tech Lead, DevOps	<code>/bmad-testarch-nfr</code> (gate) ● NFR Gate REC	Rendimiento, seguridad, fiabilidad y escalabilidad. Firma final: PASS, CONCERNS, WAIVED o FAIL.

2.5 Total estimado

RECOMENDACIÓN LIDR

En proyectos nuevos típicos, las Fases 1 a 3 ocupan unas **10–14 sesiones de dos horas**; la Fase 4 es ciclo continuo y dura lo que dure el proyecto. La estimación es nuestra, basada en sesiones reales con equipos: el método no fija una duración objetivo. Si el equipo es más pequeño o más experimentado, el rango se acorta; si el alcance es muy regulado o el dominio nuevo, se alarga.

Total proyecto greenfield: **de 2 h (Fases 1–3) + ciclo continuo (Fase 4) · cifra**
orientativa, no contractual

3 Flujo brownfield — proyecto existente

Cuando el código ya existe, BMAD se incorpora en dos pasos: primero se documenta lo que hay, después se elige el camino según la magnitud del cambio que se quiere abordar.

3.1 Paso 0 — documentar lo existente

Sesión / Roles	Workflow	Conocimiento que necesita
Sesión 1 (2 h) Roles: Tech Lead, Dev senior	<code>/bmad-document-project</code> ● Document Project OPC	Acceso al repositorio, decisiones históricas, deuda técnica conocida.
Sesión 1 Roles: Tech Lead	<code>/bmad-generate-project-context</code> ● Generate Project Context OPC	Proyecto documentado.

3.2 Después: ¿qué camino tomar?

El siguiente árbol resume cómo se elige el camino en función del tipo de cambio que el proyecto necesita.

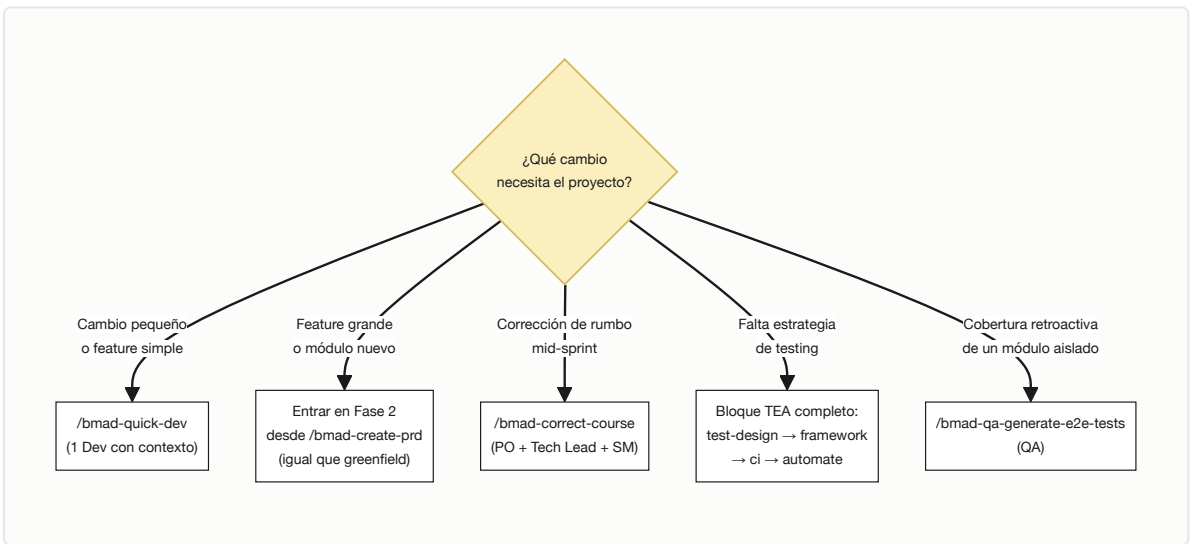


Figura 3.1 — Árbol de decisión para elegir el camino tras documentar el proyecto.

Escenario	Camino BMM	CIS / TEA aplicable	Quién
Cambio pequeño o feature simple	<code>/bmad-quick-dev</code>	—	Un Dev con contexto del proyecto.
Feature grande o módulo nuevo	Entrar en el flujo de Fase 2 (PRD) en adelante, igual que en greenfield.	● Brainstorming si la feature aún no está clara. ● Todo el bloque TEA aplica.	Mismos roles que las Fases 2–4.
Corrección de rumbo mid-sprint	<code>/bmad-correct-course</code>	● Problem Solving si hay conflictos de dirección.	PO, Tech Lead, SM.

↓ continúa en la página siguiente

Falta estrategia de testing	—	● Test Design → Framework → CI → Automation.	QA, Tech Lead.
Cobertura retroactiva en módulo aislado	—	● <code>/bmad-qa-generate-e2e-tests</code> .	QA.

TEA EN PROYECTOS BROWNFIELD

Si el proyecto existente no tiene tests o tiene cobertura baja, el bloque TEA completo (Design → Framework → CI → Automation) puede ejecutarse de forma independiente sin pasar por las fases BMM. Es la única secuencia del método que admite ejecución autónoma.

Total proyecto brownfield: de onboarding + lo que corresponda según el camino elegido.

4 Paralelismo y secuencialidad

Referencia rápida para planificar sesiones y distribuir trabajo entre varias personas. El principio: **respetar las dependencias declaradas es obligatorio; el paralelismo entre cadenas independientes es oportunidad, no obligación.**

4.1 Lo que se puede hacer en paralelo

Grupo paralelizable	Workflows	Condición o prerequisite
Fase 1 completa	brainstorming, product-brief, market-research, domain-research, technical-research	Ninguno depende de otro. Diferentes expertos pueden trabajar en paralelo.
Fase 2 después del PRD	validate-prd create-ux-design testarch-nfr (setup)	Los tres parten del PRD terminado y son independientes entre sí. Si el equipo tiene UX, QA y Tech Lead dedicados, las tres líneas avanzan a la vez.
Fase 3 línea BMM y línea TEA	(create-epics-and-stories → check-readiness) (test-design → framework → ci)	Ambas líneas necesitan la arquitectura terminada. Avanzan en paralelo si el equipo tiene PO+Tech Lead para la línea BMM y QA+DevOps para la línea TEA.
Fase 4 historias del mismo sprint	Story A ciclo completo Story B ciclo completo Story C...	Devs distintos por historia y <i>sin conflictos de código compartido</i> . Es el paralelismo nativo de Agile.
Cierre de épica con NFR check	(test-review → trace → retrospective) <i>secuencial</i> · nfr (check) cualquiera del trío	El trío de cierre es secuencial: la retro consume la puntuación y la decisión de gate como entradas. NFR check (si se invoca aquí) sí puede paralelizarse con cualquiera del trío.

PARALELISMO LÓGICO VS DECLARADO

En la Fase 2 después del PRD, el método no declara paralelismo formal. Lo que declara es que las tres skills *tienen el mismo prerequisite* (PRD terminado) y ninguna depende de las otras dos. Esa equivalencia es la que habilita el paralelismo lógico cuando el equipo tiene los roles disponibles.

4.2 Lo que NO se puede hacer en paralelo

Cadena estricta (secuencial)	Por qué es secuencial
create-prd → validate-prd → edit-prd	Cada paso valida o corrige el anterior. Adelantar no aporta nada porque no hay sobre qué validar.
create-prd → create-ux-design	Dependencia declarada: la UX no puede diseñarse sin PRD.
create-architecture → create-epics-and-stories → check-implementation-readiness	Las épicas necesitan arquitectura; el check audita las épicas.
testarch-test-design → testarch-framework → testarch-ci	El framework implementa el plan del test-design; la pipeline usa el framework instalado.
create-story → testarch-atdd → dev-story → testarch-automate → code-review	Ciclo TDD red/green más expansión de cobertura más gate final. Orden estricto.
testarch-automate → testarch-test-review → testarch-trace	Test Review audita los tests producidos por Test Automation; Traceability verifica la cobertura sobre el suite ya revisado.
testarch-nfr (setup) → testarch-nfr (check) → testarch-nfr (gate)	Cada momento usa los umbrales fijados en el anterior como baseline.
sprint-planning → create-story	Las historias se crean a partir del plan del sprint.

DEPENDENCIAS DECLARADAS Y DEPENDENCIAS LÓGICAS

El método declara dependencias explícitas en su instalación (campo `after`). Hay otras dependencias *lógicas* que no están declaradas pero el método espera que el equipo respete: **(1)** `create-architecture` sin PRD = diseño en el vacío; **(2)** `testarch-atdd` sin framework = pseudocódigo no ejecutable; **(3)** `testarch-nfr (check)` sin observabilidad = revisión sin datos reales. Conviene respetar ambas.

NOTA SOBRE DEV-STORY Y TESTARCH-AUTOMATE

La dependencia declarada de `testarch-automate` es `testarch-atdd`, no `dev-story`. En la práctica, el orden recomendado es ATDD (red) → dev-story (green) → automate (expansión de cobertura): ATDD genera tests fallidos, dev-story los hace pasar y automate añade unitarios y casos de borde. Si se invoca automate antes de dev-story, los tests recién creados encuentran el código vacío y fallan, lo que ensucia la pipeline. La declaración del método es flexible; el orden recomendado es el descrito.

5 Roles del equipo

El flujo del proyecto distribuye el trabajo entre varios roles. La tabla siguiente resume cuándo es imprescindible cada uno y qué conocimiento aporta. La sección 5.2 explica las agrupaciones típicas en equipos pequeños.

5.1 Tabla de roles

Rol	Cuándo es imprescindible	Qué debe saber
Product Owner	Fases 1–3, Sprint Planning.	Visión de negocio, usuarios objetivo, prioridades, presupuesto.
Tech Lead / Architect	Fases 2–4, bloque TEA (Test Design, CI).	Stack tecnológico, restricciones técnicas, patrones de diseño, costes de ejecución.
UX Designer	Fase 2 si el producto tiene UI.	Flujos de usuario, sistema de diseño, accesibilidad.
QA	Fase 3 (bloque TEA), Fase 4 (ATDD, automation, reviews, NFR).	Estrategia de testing, herramientas de automatización, criterios de calidad por riesgo.
Scrum Master	Fase 4 (ciclo continuo).	Velocidad del equipo, capacidad, impedimentos, gestión del backlog.
Desarrollador(es)	Fase 4.	Código, arquitectura, historias asignadas, patrones del proyecto.
DevOps / Platform / SRE	Fase 1 (technical research, opcional), Fase 2 (NFR setup, consultor), Fase 3 (arquitectura y CI Setup), Fase 4 (NFR check y gate).	Infraestructura, integración y despliegue continuos, pipelines, entornos de test, observabilidad, capacidad, plan de rollback, cumplimiento operacional.
Stakeholder / Sponsor	Fase 1, validaciones clave.	Presupuesto, objetivos de negocio, fechas críticas.

NOMENCLATURA DE DEVOPS

«DevOps» aquí es un término paraguas. Equivalentes habituales: Platform Engineer, Site Reliability Engineer (SRE), Infrastructure Engineer, Release Engineer, Cloud Engineer. En equipos pequeños, suele estar absorbido por el Tech Lead. Lo importante es identificar quién opera la infraestructura, las pipelines de integración y despliegue, los entornos, la observabilidad y los releases.

RECOMENDACIÓN LIDR

En equipos que adoptan TEA, recomendamos que DevOps participe activamente en CI Setup y en NFR Gate, aunque el método los marque como opcionales. La pipeline y los umbrales no funcionales son los puntos donde infraestructura, equipo de calidad y producto deben hablar el mismo idioma; si DevOps llega tarde, los criterios se acuerdan sin contar con la realidad de la infraestructura y se ajustan a posteriori, lo que retrasa el release.

5.2 Una persona, varios roles

La tabla anterior lista *roles funcionales*, no personas. En equipos pequeños o medianos es habitual que varios roles los cubra la misma persona:

- El **Tech Lead** suele cubrir también: facilitador del método, Architect (si no hay arquitecto dedicado) y DevOps (si no hay Platform Engineer).
- El **Product Owner** suele cubrir también: Scrum Master (cuando no hay SM dedicado) y UX Designer (cuando producto lidera la UX).
- Cuando una tabla del libro lista «PO + SM» como participantes distintos, pueden ser la misma persona cambiando de sombrero.

VISIÓN LIDR

En nuestras adopciones llamamos **BMAD Master** a la persona que facilita la introducción del método en el equipo: alguien con dominio suficiente del proceso para guiar las primeras sesiones y resolver dudas en directo. No es un agente del método ni una skill: es un rol humano que LIDR define para acompañar la curva de adopción. En equipos con un Tech Lead experimentado, el BMAD Master puede coincidir con el Tech Lead.

6 Mapeo a sesiones operativas

Si el equipo trabaja con el calendario comprimido del Libro 3 (BMAD Plan de Sesiones), las fases de este libro se agrupan en seis sesiones operativas (S1–S6).

VISIÓN LIDR El calendario S1–S6 es nuestro framing para llevar el método a equipos en formato taller. El método oficial no prescribe sesiones; deja la planificación a cada equipo. Lo que sigue resume cómo hemos visto que encajan las cuatro fases en sesiones reales.

Sesión	Qué cubre del flujo	Notas
S1	Fase 1 + Fase 2 comprimidas (research, brief, PRD).	NFR Setup ocurre al final, como cierre de S1.
S2	Primer tramo de la Fase 3: arquitectura y arranque del bloque TEA.	Tras S2, el equipo dispone de un «toolkit» de skills para correcciones asíncronas (<code>edit-prd</code> , <code>correct-course</code>) sin necesidad de convocar nueva sesión.
S3	Cierre de Fase 3 e inicio de Fase 4: épicas, sprint planning, primera historia y ATDD.	El equipo termina con la primera historia lista para implementar.
S4	Transición operativa a la implementación.	RECOMENDACIÓN LIDR En esta sesión recomendamos cerrar tres decisiones operativas obligatorias (herramienta principal, estrategia de ramas, flujo de estados de la historia) y una opcional (uso de <i>worktrees</i> o multi-agente si el equipo trabaja con Claude Code). Sin estas decisiones persistidas en <code>project-context.md</code> , cada skill aplica defaults diferentes.
S5	Fase 4 en modo continuo: ciclo iterativo por historia.	RECOMENDACIÓN LIDR Recomendamos planificar al menos dos épicas : épica 1 dedicada a configuración e infraestructura y épica 2 en adelante para features de negocio. Forzar features de negocio en la primera épica suele saltarse el setup TEA.
S6	Cierre formal de la primera épica: <code>test-review</code> → <code>trace</code> → <code>retrospective</code> .	A partir de la segunda épica, el equipo ejecuta el cierre en autonomía sin sesión facilitada.

SESIÓN S0 (OPCIONAL)

El Libro 3 incluye una sesión preliminar S0 para equipos que adoptan disciplina de testing por primera vez (`/bmad-teach-me-testing`). No forma parte del flujo del proyecto: es preadopción, no fase. El detalle vive en el Libro 3.

El detalle por sesión (duración exacta, agenda, participantes y entregables) vive en el **Libro 3 — BMAD Plan de Sesiones**.

7 Skills fuera del flujo principal

Este libro cubre el **flujo del proyecto**: las skills que se ejecutan en orden para llevarlo de la idea al release. El método incluye otras skills útiles que no encajan en una fase concreta porque resuelven necesidades transversales o paralelas. Las listamos para evitar la falsa impresión de que no existen.

Skill	Cuándo se invoca	Qué hace
/bmad-prfaq ● Working Backwards	Antes de la Fase 1, alternativa al brief.	Forja el concepto de producto mediante el formato Working Backwards (press release imaginado y FAQ). Útil cuando la idea aún no está validada y el equipo quiere ponerla a prueba antes de invertir en research.
/bmad-edit-prd ● Edit PRD	Tras Fase 2, en cualquier momento.	Modifica un PRD existente (cambios menores de alcance, refinamiento de requisitos). No reescribe; aplica edits dirigidos.
/bmad-correct-course ● Correct Course	Mid-sprint, cuando aparece desviación.	Gestiona cambios significativos durante la implementación: drift entre arquitectura y PRD, cambio de prioridades, ajuste de alcance.
/bmad-sprint-status ● Sprint Status	En cualquier momento del sprint.	Resume el estado del sprint y enruta al siguiente workflow recomendado (qué historia toca, en qué fase del ciclo está cada una).
/bmad-checkpoint-preview ● Checkpoint Preview	Antes de un merge o release.	Revisión asistida humano-en-el-bucle para entender un cambio: propósito, contexto, detalles. Útil cuando el revisor tiene poco contexto.
/bmad-customize ● Customize	Anytime.	Añade hechos persistentes, cambia plantillas, inserta hooks de activación o personaliza menús. Escribe los overrides en _bmad/custom/ y verifica el merge.
/bmad-cis-* ● Pensamiento creativo	Cuando el equipo se atasca.	Cuatro workflows del módulo creativo: design thinking (empatía con el usuario), problem solving (resolución estructurada), innovation strategy (enfoque disruptivo) y storytelling (narrativa). Ninguno es prerequisite de otra fase.
Agentes con persona Mary, John, Winston, Sally, Amelia, Paige, Murat, Sophia, Maya, Carson, Dr. Quinn, Victor, Caravaggio	Anytime.	Trece agentes especializados (analyst, PM, architect, UX, dev, tech writer, test architect y seis del módulo creativo). Útiles para conversación abierta en su área de experiencia, sin estructura de workflow.

CATÁLOGO COMPLETO

El catálogo exhaustivo de las 65 skills del método (con propósito, prerequisites, salidas, sub-pasos y duración estimada) vive en el **Libro 4 — BMAD Catálogo de Skills**. Este libro solo cubre el flujo del proyecto.

8 Recomendaciones operativas

Buenas prácticas transversales al flujo. Aplican a cualquier proyecto, greenfield o brownfield.

8.1 Contexto persistente del proyecto

PROJECT-CONTEXT.MD COMO CONTRATO DEL EQUIPO

Muchas skills (`create-story` , `dev-story` , `sprint-status` , `code-review` y varias del bloque TEA) leen `project-context.md` como punto de partida cuando se invocan en una conversación nueva. Las decisiones operativas que conviene persistir ahí: herramienta principal del equipo, estrategia de ramas, flujo de estados de la historia y, opcionalmente, uso de *worktrees* o multi-agente si se usa Claude Code. Sin esto, cada skill aplica defaults distintos y las convenciones quedan solo en la memoria del equipo.

8.2 Ventana fresca por workflow

UNA CONVERSACIÓN POR SKILL

Cada workflow se ejecuta en una nueva conversación o contexto fresco para evitar contaminación. Mezclar dos skills en la misma ventana arrastra suposiciones de la primera a la segunda y produce resultados difusos.

8.3 Validación cruzada entre LLMs

DIFERENTE LLM PARA VALIDAR

Para los workflows de validación (`/bmad-validate-prd` , `/bmad-check-implementation-readiness`) y para el code review, usa un LLM distinto al que generó el artefacto que se valida. Ganas objetividad: el LLM original tiende a confirmar sus propios huecos.

8.4 Pensamiento creativo no bloquea

CIS COMO HERRAMIENTA DEL FACILITADOR

Ninguna skill del módulo creativo es prerequisite de otra fase. Son herramientas que el facilitador activa según la dinámica del equipo. Si la sesión va con tiempo limitado, se omiten sin impacto en el flujo.

9 Verificación y fuentes

Fecha de publicación	2026-05-07
Versión BMAD verificada	BMAD v6.6.0 (Core 6.6.0 · BMM 6.6.0 · BMB 1.7.0 · CIS 0.2.0 · TEA 1.15.1)
Origen de las dependencias	Campo <code>after</code> de <code>_bmad/_config/bmad-help.csv</code> — instalación local del proyecto.
Fuentes consultadas	Repositorios oficiales <code>bmad-code-org/bmad-method</code> , <code>bmad-code-org/bmad-method-test-architecture-enterprise</code> , <code>bmad-code-org/bmad-builder</code> , <code>bmad-code-org/bmad-module-creative-intelligence-suite</code> .